# Autómata del Boletín 05, ejercicio 4: producto de dos matrices

**Caso práctico**

**Carlos** tiene que hacer una página que multiplique dos matrices. Con el enunciado delante ve que no es un formulario, sino **dos seguidos**: primero hay que preguntar las dimensiones y, con ellas, dibujar las parrillas para escribir los números. **María** le avisa: —Entre el primer formulario y el segundo, la página tiene que recordar las dimensiones. Y lo que recuerda la página, el usuario lo puede cambiar.

## 1.- Enunciado

Desarrolla un script que, utilizando las funciones de los ejercicios 2 y 3 del boletín, solicite al usuario 2 matrices, compruebe que tienen dimensiones compatibles para calcular el producto matricial y, si cumplen las condiciones, lo muestre. Si no, mostrará un mensaje de error.

- Ejercicio 2 (*introducirMatriz*): solicitar las dimensiones de una matriz n x m y generar una parrilla (una tabla HTML con un campo de texto en cada celda) para introducir los datos; validar que sean números (enteros o *float*).
- Ejercicio 3 (*productoMatricial*): producto de dos matrices de dimensiones m x n y n x t (1 ≤ n, m, t ≤ 10).

Llamaremos **m x n** a las dimensiones de A y **p x q** a las de B: se pueden multiplicar si **n = p**, y el resultado es una matriz m x q.

## 2.- Análisis

- **Eventos entrantes.** *Enviar* (el mismo botón en los dos formularios: el **estado** dice cuál de los dos se está enviando) y la **carga** sin parámetros, que se produce también con el enlace "Empezar de nuevo".
- **Estados.** Cuatro: ST0 carga; ST1 esperando las dimensiones; ST2 esperando los datos de A y B; ST3 producto mostrado. Aquí sí hacen falta estados distintos: con el mismo botón, la página hace cosas diferentes según lo que esté esperando.
- **Variables de estado.** El estado y las **cuatro dimensiones**, que se guardan en campos ocultos al pasar a ST2 (el segundo formulario no las pide, pero las necesita para leer las parrillas).
- **Predicados.** Para el primer formulario: las cuatro dimensiones son enteros entre 1 y 10 (PL0) y son compatibles (PL1). Para el segundo: todas las celdas de A (PL2) y de B (PL3) son números.
- **Datos ocultos manipulados.** En T3 se **vuelven a comprobar** PL0 y PL1 con las dimensiones ocultas. Si no se cumplen, alguien ha tocado la página: se vuelve al principio.

## 3.- Diagrama de estado-transición ampliado

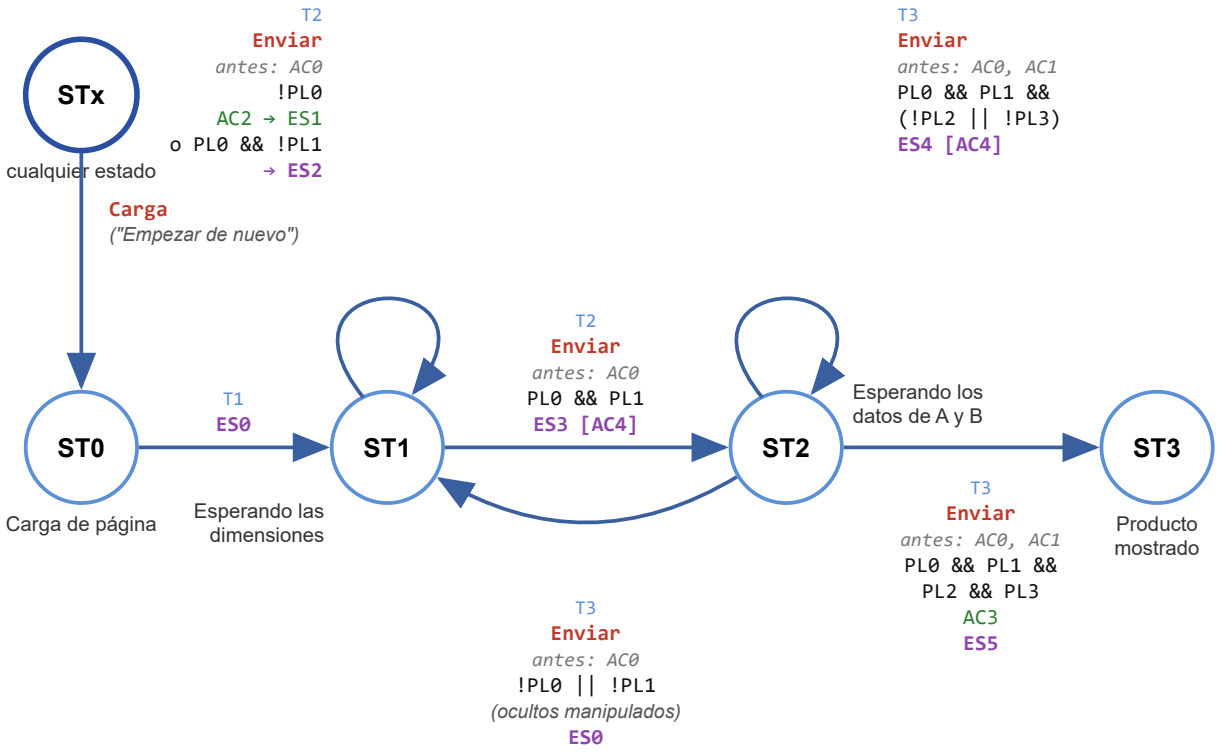



*Figura 1. Diagrama del Boletín 05, ejercicio 4*

# 4.- Autómata

## 4.1.- Tabla de descriptores

| Tipo | Abreviatura | Nombre | Significado |
|---|---|---|---|
| Evento entrante | EE0 | Carga | página sin parámetros (primera vez o enlace "Empezar de nuevo") |
| Evento entrante | EE1 | Enviar | el usuario pulsa "Enviar" |
| Evento saliente | ES0 | formularioDimensiones | formulario de las cuatro dimensiones, vacío |
| Evento saliente | ES1 | dimensionesConErrores | lo escrito y un aviso junto a cada dimensión no válida |
| Evento saliente | ES2 | dimensionesIncompatibles | "No se pueden multiplicar: A tiene n columnas y B tiene p filas" |
| Evento saliente | ES3 | parrillas | una parrilla m x n para A y otra p x q para B, vacías |
| Evento saliente | ES4 | parrillasConErrores | las parrillas con lo escrito y las celdas no válidas en rojo |
| Evento saliente | ES5 | resultado | A, B y A x B, con el enlace "Empezar de nuevo" (sin botón Enviar) |
| Estado | ST0 | CargaPagina | la página aún no se ha mostrado |
| Estado | ST1 | EsperandoDimensiones | se muestra el formulario de dimensiones |
| Estado | ST2 | EsperandoDatos | se muestran las parrillas de A y B |
| Estado | ST3 | ProductoMostrado | se muestra el resultado |
| Variable de estado | VE0 | `$estado` | estado actual (campo oculto) |
| Variable de estado | VE1–VE4 | `$m`, `$n`, `$p`, `$q` | dimensiones (campos ocultos en ST2) |
| Variable | VE5, VE6 | `$a`, `$b` | las matrices (arrays de 2 dimensiones) |
| Variable | VE7 | `$c` | el producto |
| Predicado | PL0 | dimensiones válidas | m, n, p y q son enteros entre 1 y 10 |
| Predicado | PL1 | compatibles | n = p (columnas de A = filas de B) |
| Predicado | PL2 | A numérica | todas las celdas de A son números |
| Predicado | PL3 | B numérica | todas las celdas de B son números |
| Acción | AC0 | leer dimensiones | `trim` y `FILTER_VALIDATE_INT` (1–10) de m, n, p, q (del formulario o de los ocultos) |
| Acción | AC1 | leer matrices | leer `a[i][j]` y `b[i][j]`, admitir coma decimal, `FILTER_VALIDATE_FLOAT` |
| Acción | AC2 | avisos de dimensiones | un aviso por cada dimensión no válida |

| Tipo | Abreviatura | Nombre | Significado |
|---|---|---|---|
| Acción | AC3 | `$c = productoMatricial($a, $b)` | c[i][j] = suma de a[i][k] · b[k][j] |
| Acción | AC4 | guardar dimensiones | escribir m, n, p, q en campos ocultos (al generar ES3 y ES4) |

### 4.2.- Tabla de eventos-estados ampliada

| | Carga (EE0) | Enviar (EE1) |
|---|---|---|
| ST0 | T1 | T1 |
| ST1 | T1 | T2 |
| ST2 | T1 | T3 |
| ST3 | T1 | --- |

En ST3 la página no tiene botón *Enviar* (`---`): si llega, se ha manipulado y el programa empieza de nuevo (T1).

### 4.3.- Tabla de transiciones

| # | AEs antes de los PLs | PLs | Nuevo ST | AEs después | ESs |
|---|---|---|---|---|---|
| T1 | --- | --- | ST1 | --- | ES0 |
| T2 | AC0 | `!PL0` | ST1 | AC2 | ES1 |
| T2 | AC0 | `PL0 && !PL1` | ST1 | --- | ES2 |
| T2 | AC0 | `PL0 && PL1` | ST2 | --- | ES3 [AC4] |
| T3 | AC0 | `!PL0 \|\| !PL1` | ST1 | --- | ES0 |
| T3 | AC0, AC1 | `PL0 && PL1 && (!PL2 \|\| !PL3)` | ST2 | --- | ES4 [AC4] |
| T3 | AC0, AC1 | `PL0 && PL1 && PL2 && PL3` | ST3 | AC3 | ES5 |

En T3 la acción AC1 (leer las parrillas) solo se hace si se cumplen PL0 y PL1: sin dimensiones válidas no se sabe cuántas celdas leer.

### 4.4.- Comprobación de predicados

T2 tiene dos predicados (PL1 no se evalúa si PL0 es falso):

| PL0 | PL1 | `!PL0` | `PL0 && !PL1` | `PL0 && PL1` |
|---|---|---|---|---|
| 0 | x | 1 | 0 | 0 |
| 1 | 0 | 0 | 1 | 0 |
| 1 | 1 | 0 | 0 | 1 |

T3 tiene cuatro, pero PL2 y PL3 solo se evalúan si PL0 y PL1 son ciertos:

| PL0 | PL1 | PL2 | PL3 | `!PL0 \|\| !PL1` | `... && (!PL2 \|\| !PL3)` | `... && PL2 && PL3` |
|---|---|---|---|---|---|---|
| 0 | x | x | x | 1 | 0 | 0 |
| 1 | 0 | x | x | 1 | 0 | 0 |
| 1 | 1 | 0 | 0 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 | 0 | 1 | 0 |
| 1 | 1 | 1 | 0 | 0 | 1 | 0 |
| 1 | 1 | 1 | 1 | 0 | 0 | 1 |

## 5.- Del autómata al código

Fragmento de `solucion/index.php`. Observa que el evento imposible (`---`) se guarda en la tabla como `null` y se trata como T1:

```
$automata = [
    ST0 => [EE0 => 1, EE1 => 1],
    ST1 => [EE0 => 1, EE1 => 2],
    ST2 => [EE0 => 1, EE1 => 3],
    ST3 => [EE0 => 1, EE1 => null],       // en ST3 no hay botón Enviar
];
$transicion = $automata[$estado][$evento] ?? 1;           // evento imposible: empezar de
nuevo

switch ($transicion) {
    // ...
    case 3:                                               // ---- T3
        [, $d] = leerDimensiones($_GET);                  // AC0 (de los campos ocultos)
        $pl0 = !in_array(false, $d, true);
        if (!$pl0 || $d['n'] !== $d['p']) {               // !PL0 || !PL1: ocultos
manipulados
            $estado = ST1;
            es0();
            break;
        }
        [$textosA, $numerosA] = leerMatriz($_GET['a'] ?? null, $d['m'], $d['n']);   // AC1
        [$textosB, $numerosB] = leerMatriz($_GET['b'] ?? null, $d['p'], $d['q']);
        $pl2 = todasNumeros($numerosA);
        $pl3 = todasNumeros($numerosB);
        if (!$pl2 || !$pl3) {                             // PL0 && PL1 && (!PL2 || !PL3)
            $estado = ST2;
            es3($d, $textosA, $numerosA, $textosB, $numerosB);  // ES4: valores escritos y
celdas en rojo
        } else {                                          // PL0 && PL1 && PL2 && PL3
            $estado = ST3;
            $c = productoMatricial($numerosA, $numerosB);  // AC3
            es5($numerosA, $numerosB, $c);
        }
        break;
}
```

**Debes conocer**

`leerMatriz()` no se fía de lo que llega: comprueba que `$_GET['a']` es un array y que cada celda existe y es un texto antes de validarla. Si alguien envía `a=hola` en lugar de `a[0][0]=...`, las celdas que faltan cuentan como no válidas (camino ES4) y no se produce ningún aviso de PHP.

## 6.- Pruebas: un camino por cada fila

| Prueba | Camino | Resultado comprobado |
|---|---|---|
| Primera carga | T1 | formulario de dimensiones; `estado` = 1 |
| m = 0, q = "abc" | T2: `!PL0` | avisos junto a m y q |
| A 2 x 3, B 2 x 2 | T2: `PL0 && !PL1` | "A tiene 3 columnas y B tiene 2 filas" |
| A 2 x 3, B 3 x 2 | T2: `PL0 && PL1` | parrillas 2 x 3 y 3 x 2; `estado` = 2; dimensiones ocultas |
| una celda de A con "x" | T3: `!PL2` | la celda en rojo; se conservan los demás valores |
| A = [[1,2,3],[4,5,6]], B = [[7,8],[9,10],[11,12]] | T3: `PL2 && PL3` | A x B = [[58, 64], [139, 154]]; sin botón Enviar |
| A 1 x 1 con "1,5", B 1 x 1 con 2 | T3: `PL2 && PL3` | 3 (acepta la coma decimal) |
| oculto n = 3, p = 2 (manipulado) | T3: `!PL1` | vuelve al formulario de dimensiones |
| oculto m = 99 | T3: `!PL0` | vuelve al formulario de dimensiones |
| `a=hola` | T3: `!PL2` | celdas en rojo, sin avisos de PHP |
| Enviar en ST3 | `---` → T1 | formulario de dimensiones |

## 7.- Para practicar

- Añade a ES5 un botón "Cambiar los números" que vuelva a ST2 con las mismas dimensiones y los valores escritos. ¿Qué evento entrante nuevo aparece? ¿En qué celdas de la tabla de eventos-estados? ¿Qué variables de estado nuevas tiene que guardar ES5?
- ¿Y si se quiere permitir también la suma A + B? Añade un predicado "las dos matrices tienen las mismas dimensiones" y decide qué camino sigue cada combinación.

---